

Cours 18 — Données en tables : le format CSV

Semaine 18 ■ Manuel Hachette ch. 7 (§ 7.1) ■ 2 × 1 h

Objectifs

- Décrire l'organisation d'une table : enregistrements et attributs
- Lire le format CSV et le charger en liste de dictionnaires
- Convertir les champs (tout arrive en chaîne!)

1. Les données en table

Carnet de notes, liste des communes, résultats sportifs : partout des données organisées en **table** — des lignes appelées **enregistrements** (une entité chacune), des colonnes appelées **attributs** (ou champs, ou descripteurs).

prenom	jour	mois	taille
Ada	10	12	1.63
Blaise	19	6	1.71
Chloé	2	1	1.58

2. Le format CSV

CSV (*Comma-Separated Values*) : un fichier texte, un enregistrement par ligne, les valeurs séparées par des virgules (parfois des points-virgules), la première ligne donnant les noms d'attributs :

```
prenom,jour,mois,taille
Ada,10,12,1.63
Blaise,19,6,1.71
Chloé,2,1,1.58
```

C'est le format d'échange universel de l'open data et des tableurs — un simple fichier texte lisible partout.

3. Charger un CSV en Python

Représentation choisie : **une liste de dictionnaires** — un dictionnaire par enregistrement, les clés étant les noms d'attributs (tout le vocabulaire de la période 2!).

```
import csv

def charger(nom_fichier):
    """Renvoie la table (liste de dictionnaires) du fichier CSV."""
    with open(nom_fichier, encoding="utf-8") as f:
        return list(csv.DictReader(f))

table = charger("eleves.csv")
table[0] # {'prenom': 'Ada', 'jour': '10', ...}
table[0]["prenom"] # 'Ada'
len(table) # 3 enregistrements
```

Le `with open(...)` ouvre le fichier et garantit sa fermeture ; `encoding="utf-8"` : semaine 6 en action.

4. Tout arrive en chaîne !

Dans le CSV, tout est du **texte** : `table[0]["taille"]` vaut `'1.63'` (une chaîne). Sans conversion, les comparaisons mentent : `'10' < '9'` est `True` (ordre des caractères, semaine 6) ! Première étape après chargement : **convertir les champs numériques**.

```
def convertir(table):  
    """Convertit en place les champs numériques (MODIFIE table)."""  
    for e in table:  
        e["jour"] = int(e["jour"])  
        e["mois"] = int(e["mois"])  
        e["taille"] = float(e["taille"])
```

5. Valider avant d'exploiter

Les données réelles sont sales : champs vides, doublons, valeurs impossibles. Avant toute exploitation, quelques **tests de cohérence** : les mois sont-ils entre 1 et 12 ? des tailles négatives ? des enregistrements en double ? — c'est le programme de la semaine prochaine.

À retenir

- Table = enregistrements (lignes) × attributs (colonnes) ; en Python : liste de dictionnaires.
- CSV : texte, séparateur virgule/point-virgule, première ligne = en-têtes.
- Chargement : `csv.DictReader` dans un `with open(...)`.
- Tout champ arrive en **chaîne** : convertir (`int`, `float`) avant de calculer ou comparer.